

Adapted from a final Machine Learning class project; AI tools were used as a research aid during development.

Predicting Real Estate Prices with Machine Learning

What Model Comparison Revealed About a Nonlinear Problem

The Problem

Choosing a machine-learning model is rarely just a matter of running several algorithms and selecting the one with the highest score. The more important task is understanding the data well enough to know why one modeling approach works better than another.

I explored that question using a real-estate valuation dataset from Sindian District containing 414 transactions. The target variable was house price per unit area, with six predictors: transaction date, house age, distance to the nearest MRT station, number of nearby convenience stores, latitude, and longitude. I compared conventional linear regression with regularized regression, Support Vector Regression (SVR), ensemble methods, and a neural network.

Start with the Data, Not the Algorithm

Before building a model, I cleaned and explored the dataset. I removed the “No” column because it was a row index rather than a meaningful predictor. There were no missing values or duplicate records, so no imputation or de-duplication was required.

Exploratory analysis surfaced two important characteristics. Distance to the nearest MRT station was strongly right-skewed, while the target contained a high-value observation of 117.5. I retained that observation because there was no evidence it was a data error, while keeping its possible influence on model performance in mind.

Correlation analysis identified distance to an MRT station as the strongest predictor, at approximately -0.67 with price. More importantly, the scatter plot showed that price did not decline with MRT distance in a straight line. It resembled a decay curve: prices dropped rapidly as distance increased near a station, then flattened for properties already far away. That observation changed the modeling strategy.

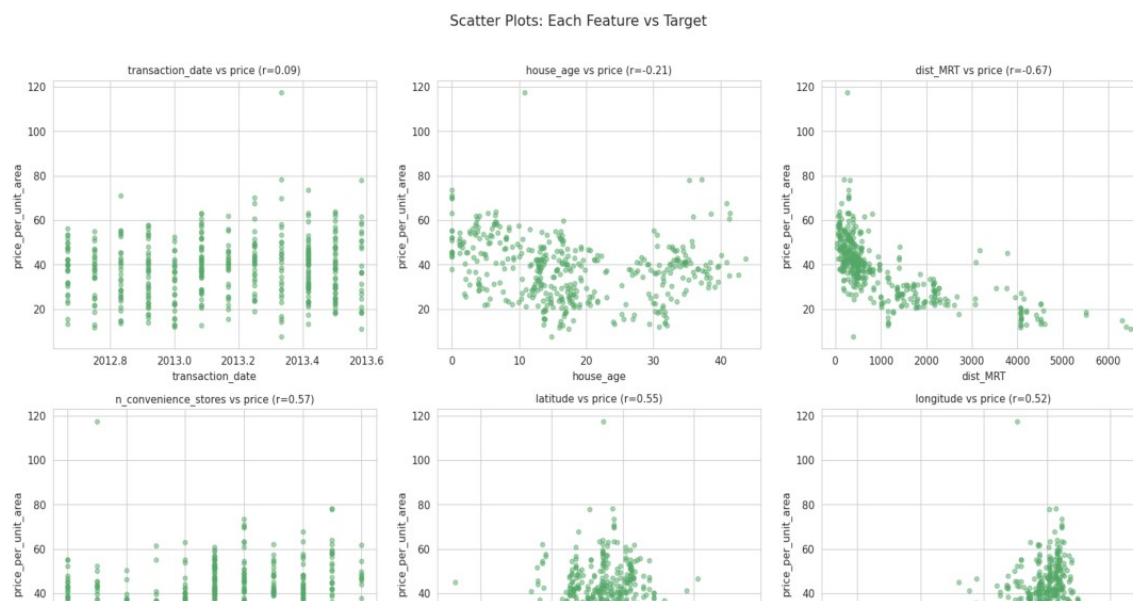


Figure 1. Feature-vs-target plots. The MRT-distance relationship is visibly nonlinear, motivating the use of nonlinear models.

Standardizing the Inputs

The six features operated on very different numerical scales. I standardized all predictors to zero mean and unit variance. This was especially important for Ridge and Lasso regression, where penalties operate on coefficient magnitude, and for SVR and the neural network, whose calculations are sensitive to feature scale. The scaler was fitted only to the training data and then applied to the test data, preventing information from the test set from leaking into model training. An 80/20 split with random seed 42 produced 331 training observations and 83 test observations.

Establishing a Linear Baseline

Ordinary least squares regression using all six features produced a test MSE of 53.506 and R^2 of 0.681. Recursive Feature Elimination selected four predictors—house age, MRT distance, number of convenience stores, and latitude—but the reduced model performed almost identically, suggesting that transaction date and longitude contributed relatively little independent linear information.

Ridge improved test MSE slightly to 52.894, while Lasso achieved 53.033. The improvement over ordinary linear regression was small, consistent with the exploratory finding that severe multicollinearity was not the central problem. The main limitation remained: these were linear models trying to describe a relationship that the data suggested was nonlinear.

Letting Nonlinear Models Capture the Pattern

I next tested SVR with a radial basis function kernel, Random Forest, Gradient Boosting, and a Multi-Layer Perceptron neural network. All four nonlinear models clearly outperformed the linear alternatives on the held-out test set. Random Forest achieved the strongest result, with a test MSE of 31.838 and R^2 of 0.810. That result was consistent with what the scatter plots had suggested earlier: the data contained relationships that a straight-line model could not adequately represent.

Model	Test MSE	Test R ²
Linear Regression	53.506	0.681
Ridge	52.894	0.685
Lasso	53.033	0.684
SVR	38.401	0.771
Random Forest	31.838	0.810
Gradient Boosting	34.003	0.797
MLP Neural Network	38.263	0.772

When the Neural Network Failed

The neural network produced another useful lesson because the first attempt did not work. An initial, larger MLP with lighter regularization badly overfit the small dataset and produced a negative test R². I reduced the network's capacity, increased L2 regularization to $\alpha=1.0$, and used the lbfgs solver, which was more appropriate for this relatively small training set. The revised model reached a test R² of approximately 0.77.

This was an important reminder that model complexity is not automatically an advantage. A model powerful enough to learn complicated patterns is also powerful enough to learn noise. With only 414 observations, controlling variance mattered.

Cross-Validation Changes the Question

A single train-test split can be misleading, particularly when a dataset contains unusual observations. For final model selection, I used 10-fold cross-validation for the principal models. Instead of asking which algorithm performed best on one particular split, cross-validation asks a more useful question: how consistently does the model perform when different subsets of the data are held out?

The analysis selected Random Forest as the best model based on cross-validated MSE, at approximately 66.5. SVR followed closely at 68.8 and Gradient Boosting at 73.0, while the linear and regularized models clustered around 86.9–88.6. The pattern therefore persisted across folds rather than depending on one favorable test split.

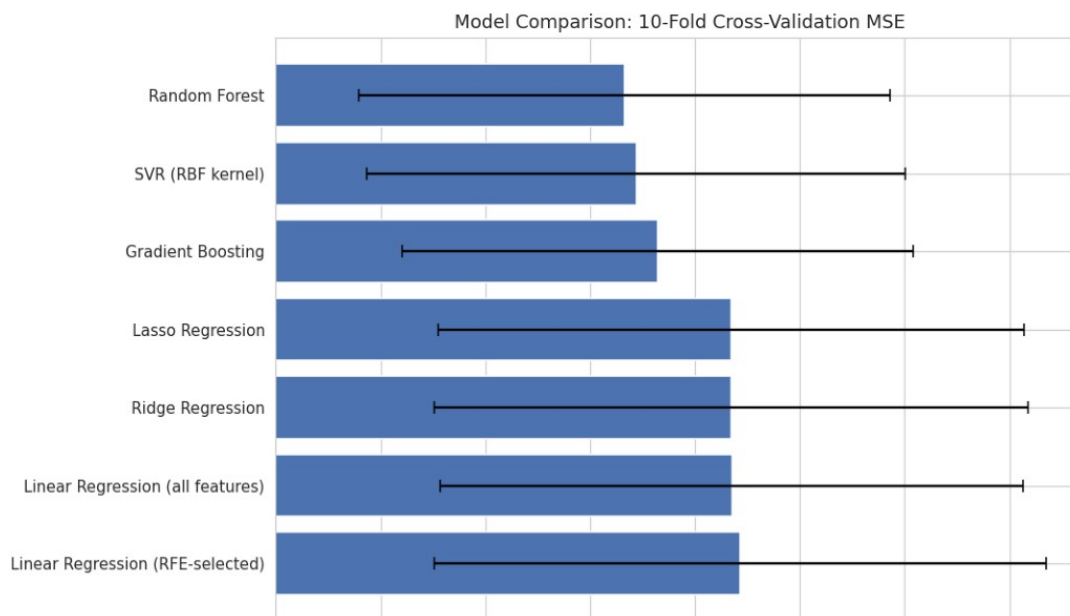


Figure 2. Ten-fold cross-validation MSE by model (lower is better). Random Forest achieved the lowest mean CV MSE.

What I Learned

The most useful lesson from this project was not that Random Forest is universally “better” than linear regression. The lesson was that model selection should follow evidence from the data.

EDA exposed the nonlinear price-distance relationship. Linear models established an interpretable baseline. Ridge and Lasso tested whether regularization could materially improve that baseline. Nonlinear methods then tested the hypothesis suggested by the scatter plots. Cross-validation provided a more robust way to choose among them. Even the unsuccessful first neural-network experiment contributed information: it demonstrated how easily a high-capacity model can overfit a small dataset and why tuning and regularization matter.

For this dataset, Random Forest provided the best balance. It was flexible enough to model nonlinear relationships while using ensemble averaging to control variance. Meanwhile, the simpler linear models remained valuable because their coefficients were easier to interpret. In applied machine learning, the best model is not necessarily the most sophisticated algorithm. It is the model whose assumptions, complexity, generalization performance, and interpretability best fit the problem being solved.

Technical stack: Python, pandas, NumPy, scikit-learn, Matplotlib; Linear Regression, RFE, Ridge, Lasso, SVR, Random Forest, Gradient Boosting, MLP, and 10-fold cross-validation.